OCP GLOBAL SUMMIT 2026 • TIME APPLIANCES PROJECT

数据中心多域时间同步

Christopher Hall · Srinivasan S. Iyengar，Intel

时长 21 分 17 秒

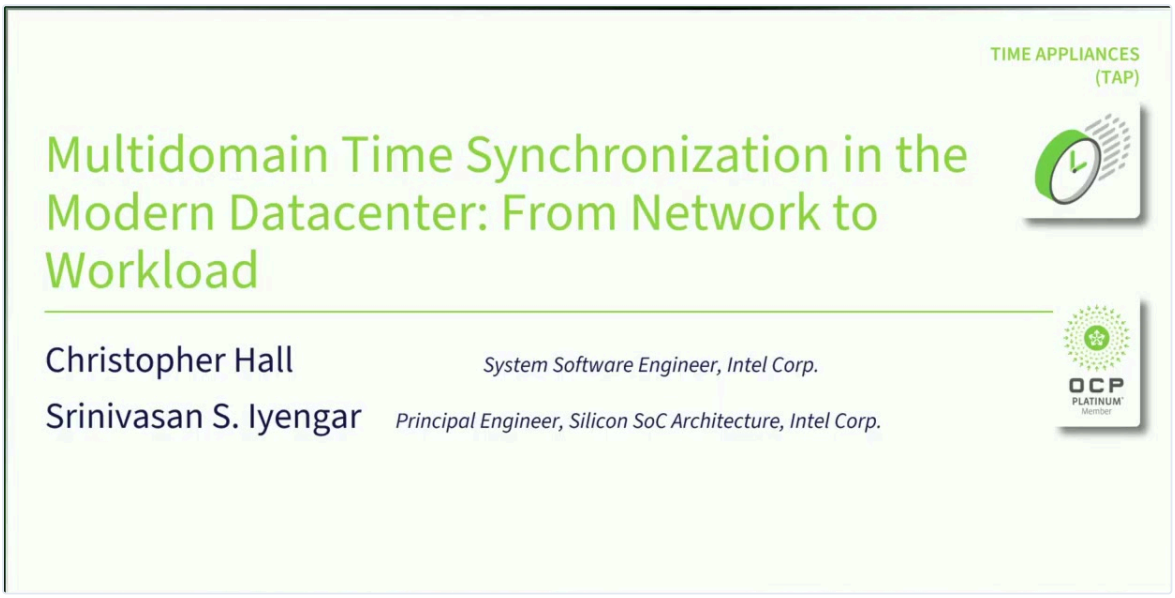
来源 youtube.com/watch?v=OQhaLXc7_PQ

页数 9 页幻灯片

数据中心多域时间同步：从网络到工作负载

> OCP Global Summit 2026 · Time Appliances Project (TAP) 分会场 讲者：Christopher Hall（Intel 系统软件工程师）、Srinivasan S. Iyengar（Intel 首席工程师，硅片 SoC 架构） 全长 21 分 17 秒 ｜ 共 9 页幻灯片

01 开场：网络架构与系统架构两条线



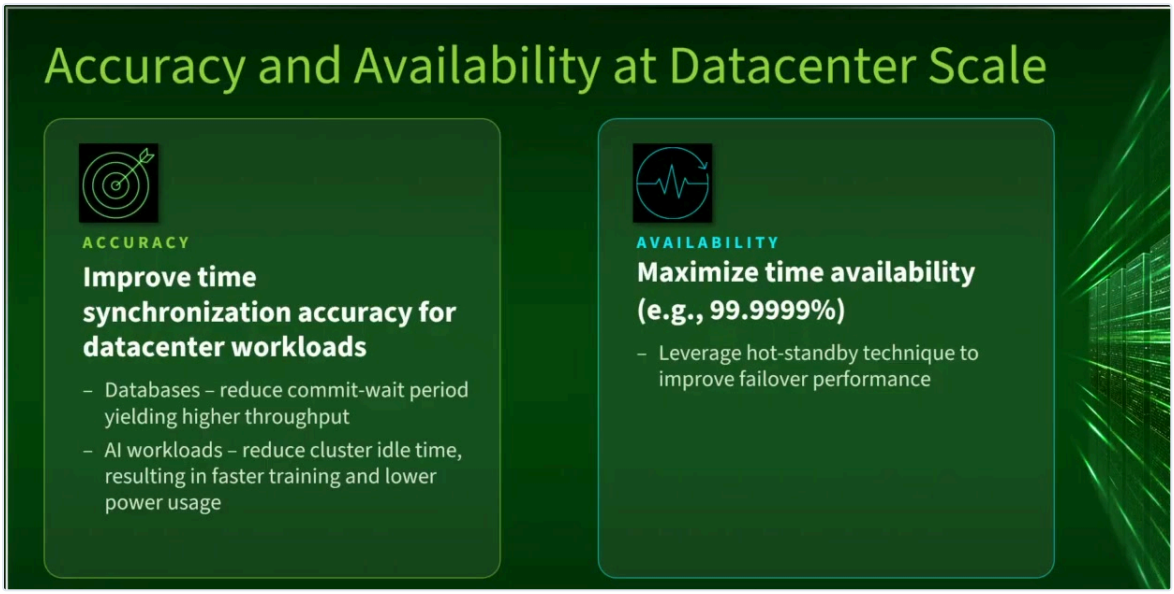
开场标题页：Multidomain Time Synchronization in the Modern Datacenter: From Network to Workload，讲者为 Christopher Hall 与 Srinivasan S. Iyengar（均为 Intel）

好。呃，那么今天，正如 Hal 刚才介绍的，我要讲的是现代数据中心里的多域时间同步（multidomain time synchronization）。呃，我会同时讲网络架构和系统架构两个方面。所以，我今天具体要讲的两件事是……

> "All right. Um so today I'm going to uh as as Hal said, I'm going to talk about multi domain time synchronization in the data center. And uh I'm going to talk about both the the network architecture and uh and the system architecture. So, the two things that um that I'm going to talk about specifically are uh"

> **小结** 议题限定为"多域"，并明确从网络侧与系统侧两条线同时展开——这也是后续所有方案的落点：网络侧缩短跳数，系统侧在主机与网卡上引入多个时间基。

02　数据中心规模下的精度与可用性



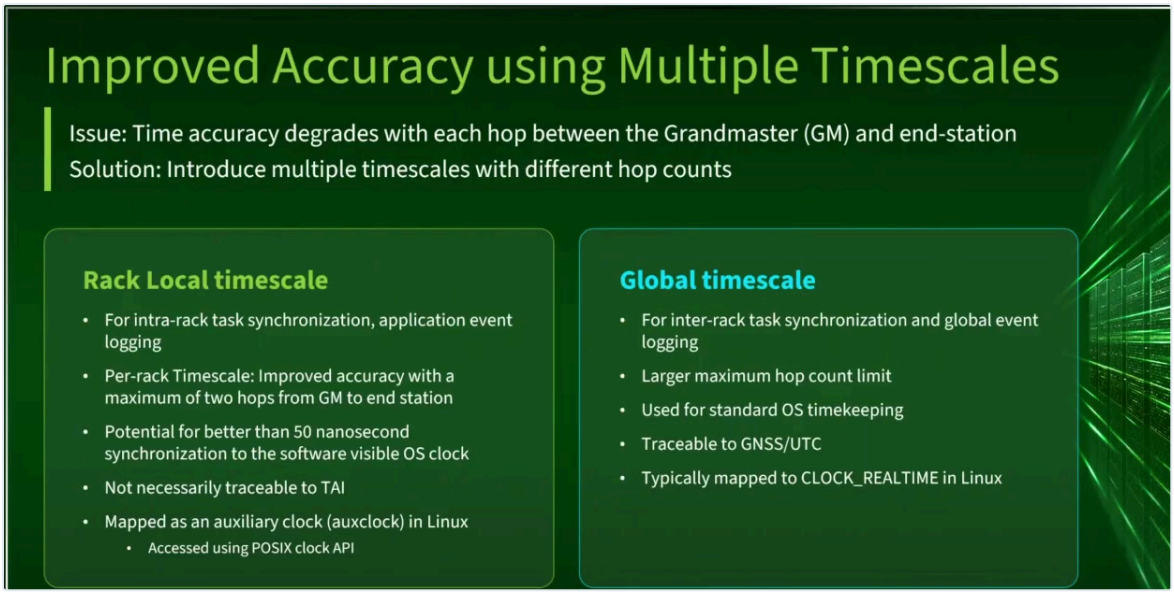
Accuracy and Availability at Datacenter Scale：左侧 ACCURACY（为数据中心负载提升时间同步精度），右侧 AVAILABILITY（把时间可用性做到 99.9999% 量级，靠热备份改善倒换性能）

呃，我们想给数据中心带来的是更高的精度和更高的可用性。呃，我想大多数人都知道时间精度是什么——就是与时钟参考之间控制在多少纳秒以内。可用性呢，可能稍微不那么直观；但当时间源消失时，也就是 GM 不见了的时候，呃，目前 PTP 的基础 profile 用的是 BMCA——BTCA 是新叫法——来切换。而切换要花一些时间。在那段时间里，我们处于保持（holdover）状态，各个时钟在互相漂移。呃，所以我们要提出一个方案，把这个绕过去。那么，首先我来讲精度的改进。呃，基本上，我觉得……

> "you know, what we want to bring to the data center are um increased accuracy and increased availability. Um I think uh most people know what time accuracy is. It's uh within whatever nanoseconds of uh of the clock reference. Availability um maybe it's a little less intuitive, but when uh when the time source goes away, she the GM, um you know, currently uh sort of the the base um the base profile of PTP uses uh BMCA beat BTCA is the the new term um to switch over. And that takes some time. And during that time, we're in we're in holdover and our clocks are drifting apart. Um so we're going to propose a solution where we uh don't uh what where we work around that. So, uh first of all, I'll talk about uh improved accuracy. Um so so basically um I I think the well,"

> **小结**　两个目标：精度（纳秒级偏差）与可用性（99.9999%）。痛点在于 GM 失效后 BMCA 倒换耗时的这段时间里系统处于保持态，各时钟相互漂移——这正是后文"热备份"要消除的窗口。

03　多时间尺度：机架本地与全局



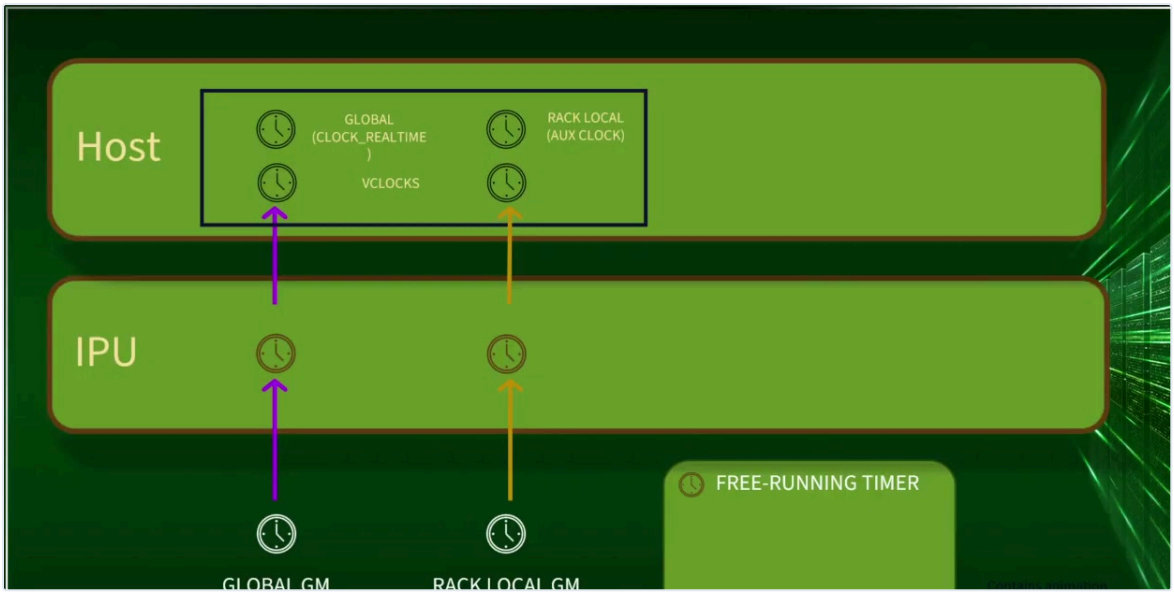
Improved Accuracy using Multiple Timescales：问题为精度随 GM 到端站的每一跳衰减，方案是引入跳数不同的多个时间尺度；左栏 Rack Local timescale，右栏 Global timescale

呃，我在这里稍微倒过来讲。全局时间尺度（global time scale）基本上就是我们现有的东西。呃，它是可溯源到 UTC/TAI 的时间。呃，在 Linux 系统上，它映射到实时时钟。呃，这是很标准的东西。那么，我们提出的是第二个时间尺度，我们把它叫做机架本地时间尺度（rack local time scale）。基本上，这就是——如果我们有一个完全包含在一个机架内的应用，那么我们就可以用一个机架本地时间尺度，它经过的跳数更少，因此在为应用做同步时会表现更好，因为跳数更少。基本上每一跳——比如说每条链路我们会损失 4 纳秒——你把它乘以三，现在你就有了三倍、三倍的误差。所以，我们在这里提出的，就是缩短从 grandmaster 过来的网络距离，以提升精度。

> "I'll I'll talk about this in slightly uh backwards order here. The uh the global time scale is is basically what we have currently. Um it's it's a UTC TAI traceable time. And uh it's uh on on the Linux system, it's mapped to the the real-time clock. Um so this is pretty standard stuff. So, uh what we're proposing is a second time scale, and we're going to call this the rack local time scale. And so basically this is going to be the each um so if we have an application that's contained completely within a rack, now we have uh we can use a rack local time scale and so that's going to be synchronized over fewer hops and it's uh going to uh do better at synchronizing applic- doing do better at synch- synchronizing the application because there will be fewer hops. Basically each hop uh we lose say for example we lose uh 4 nanoseconds per link. You know, you multiply that by three. Now you've got uh you know, three times three times that error. And uh so so basically there's uh uh what we're proposing here is to to shorten the uh the network distance from the uh grandmaster to improve accuracy."

> **小结**　全局时间（可溯源 UTC/TAI，映射 CLOCK_REALTIME）适合跨机架；机架本地时间（≤2 跳）适合机架内任务，误差按每链路约 4ns 累加，缩短距离即可显著降误差。代价是机架本地时间不必可溯源到 TAI。

04　系统架构：主机与 IPU 上的双时间基



系统架构图：Host 内含 GLOBAL（CLOCK_REALTIME）、RACK LOCAL（AUX CLOCK）与 VCLOCKS；下方 IPU 分别从 GLOBAL GM 与 RACK LOCAL GM 取时；右侧另有 FREE-RUNNING TIMER

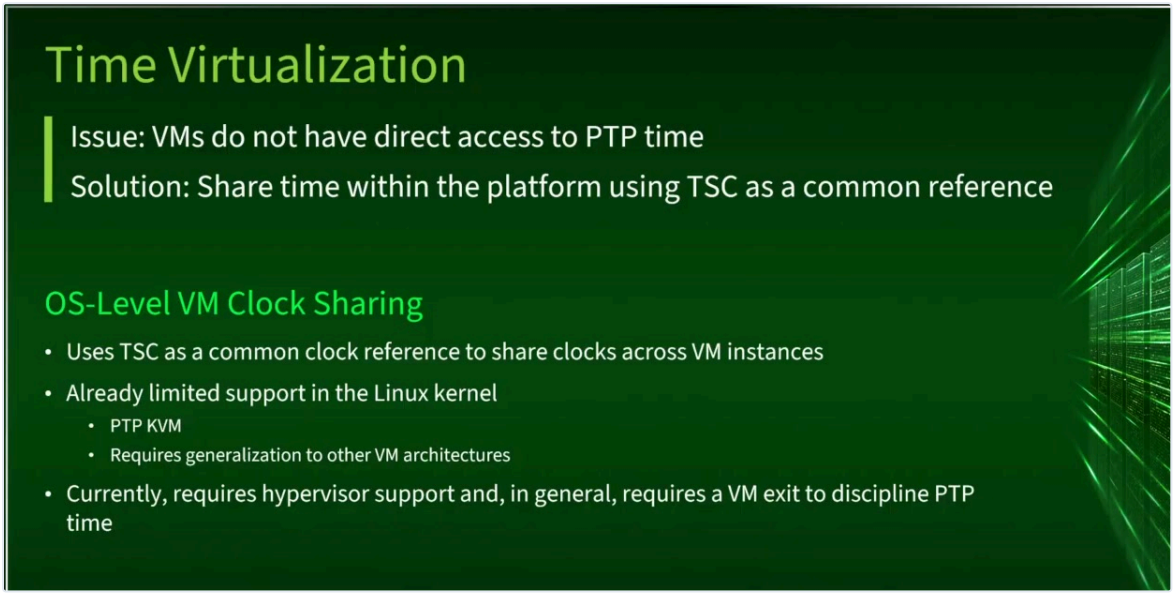
这是我所说内容的快速图示。呃，我们的网卡——我们把它标为 IPU，这是 Intel 的产品，大致上就是一块智能网卡（smart NIC）。呃，所以后面我会把 IPU 和 NIC 混着用，但图里画的就是这个。呃，我在这里展示的是：我们有一个全局时间尺度，映射到 CLOCK_REALTIME，这仍然是 PTP 使用单一时间尺度的标准方式。那么现在，我们再加上第二个时间尺度，也就是机架本地时间尺度。呃，就 Linux 系统而言，这里加了几样东西。我们有一个 V-clock。呃，这是一个——"虚拟 PTP"是个被过度使用的词——但基本上，有一个软件 PTP，负责做翻译（translation）。你还会注意到，我们在网卡里有一个自由运行时钟（free-running clock）。我们并不直接去驯服网卡时钟。呃，所有的驯服都包含在 V-clock 自身里面。这里还要注意的另一部分是 aux clock 的概念。那是 Linux 相对较新的特性，允许我们拥有几乎任意数量的时间基，呃，它们的行为——我的意思是，它不是 CLOCK_REALTIME，但它们的行为完全像 CLOCK_REALTIME，通过 POSIX 时钟接口访问。呃，这是相当新的东西，还有一些开发工作要做。呃，还有一个密切相关的话题是时间虚拟化。呃，所以虚拟机……

> "So, here's uh here's a quick graphical representation of what I'm talking about. And uh so uh our network card uh we la- label this as the IPU. This is an Intel product. It's uh it's a more or less a smart NIC. Um so it going forward I'll I'll use IPU NIC interchangeably but uh that's what the what we have in the diagram. So, uh what I'm what I'm showing here is that we have a global time scale. It's going to be mapped clock real time and again this is the standard way that PTP use is used with a single uh single time scale. So, now what we do is we add a second time scale and this is going to be the rack local time scale. And uh so there's a in terms of the Linux system, there's a couple of things that uh get added here. So, we have a V-clock. Um so this is uh going to be a board It's it virtual PhD is a uh pretty overloaded term, so uh but basically uh there's a there's a uh there's a software PhD that uh does the translation. And you'll also notice that we have the uh um have a free-running clock in the NIC. So, we're not directly uh disciplining the NIC clock. Um all the all of the disciplining is contained within the V clock itself. And the other the other part to um to notice here is that we have the the concept of the aux clock. And that's a relatively new Linux feature that allows us to uh have more or less an arbitrary number of of time bases um that are behave exactly I mean I mean it's it's not clock real time, but they behave exactly like clock real time using the POSIX uh POSIX clock interface. Um, and that that's something that's

> again pretty recent, and there's still some development uh still some development to do there. So, um and and another um another closely related topic here is time virtualization. Um, so the VMs"

> **小结**　落地机制三件套：V-clock 做软件翻译（驯服只发生在 V-clock 内，网卡时钟保持自由运行）、aux clock 提供任意多个类 CLOCK_REALTIME 时间基、IPU/NIC 作为时间入口。这样主机可在不改动硬件的前提下同时持有全局与机架本地两个时间基。

05　时间虚拟化：以 TSC 作公共基准



Time Virtualization：问题为 VM 无法直接访问 PTP 时间，方案是以 TSC 作为平台内共享时间的公共参考；下方列出 OS-Level VM Clock Sharing 的现状与限制

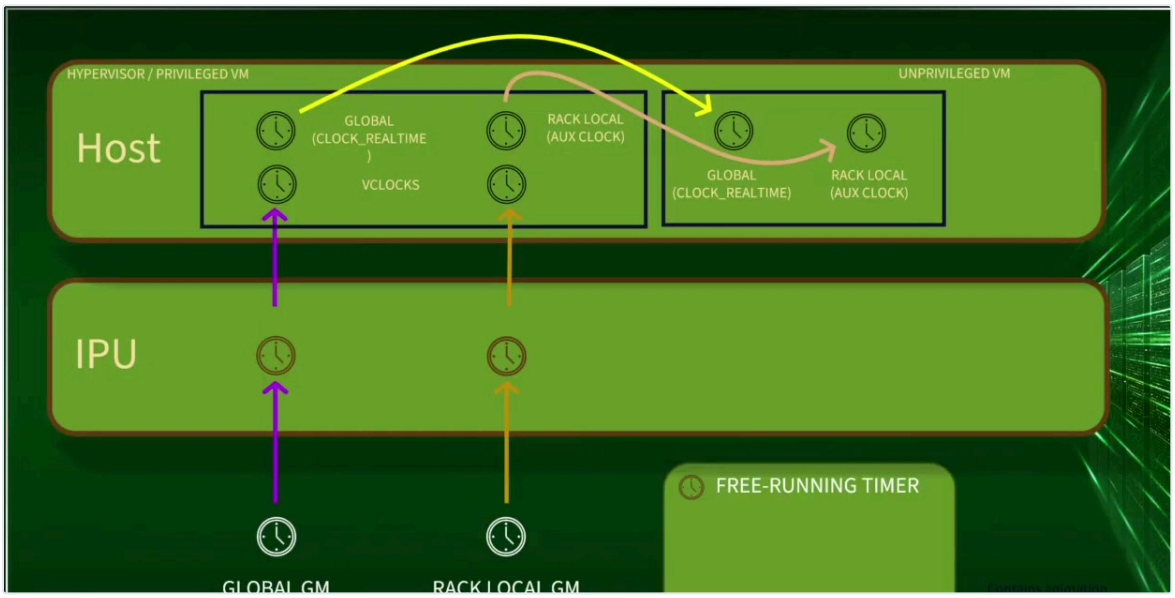
……无法直接访问——所以，非特权 VM 无法直接访问 PTP 时间。那么，解决办法是：在软件层，我们用 TSC 作为公共时间参考。呃，所有 VM 都能访问 TSC。呃，它们可以用它来把时间从一处翻译到另一处——从 hypervisor 到另一个 VM、从一个 VM 到另一个 VM，诸如此类。所以基本上，我们在 Linux 内核里已经有对这个的有限支持，呃，就 PTP 而言是 PTP KVM 驱动。呃，它基本上专门绑定 KVM，目前不能用于其他 hypervisor，但概念是一样的，只是实现代码的问题。呃，目前还有另一个问题，就是需要 VMX 相关的能力，但那大概也可以绕过去——有志者事竟成那种。呃，那我们来看一下，我们在"与虚拟机共享时间"这件事上提出的是什么。呃，这就是上一页的出发点。

> "don't have access direct access to uh so the non-privileged VMs don't have access direct access to PTP time. So, um we So, there's a basically the solution is in software, we use TSC as a common time reference. Uh all the VMs have access to TSC. Um, and uh they can use that to translate time from one uh uh, from the hypervisor to another VM, one VM to the other, that sort of thing. So, um, basically, this is we we already have limited support for this in the Linux kernel and uh, in terms of uh, PTP the PTP KVM driver. Um, so, it's basically it's it's tied specifically to KVM, doesn't currently work with other um, other uh, um, other hypervisors, but uh, it's basically the same concept and it's uh, just a matter of of implementing the code. Um, and and and currently, there's also another problem that where VMX its are are required, but that's probably something that could be work worked around as well, where there's where there's a will,

> there's a way kind of thing. So, uh, we'll we'll take a look at uh, what's what we're proposing here for uh, virtual is it sharing time uh, with a virtual machine. So, uh, here's where we started from the last slide."

> **小结**　非特权 VM 拿不到 PTP 时间，但都能读 TSC——于是把 TSC 当作公共基准，用软件做时间翻译。内核已有 PTP KVM 驱动的雏形，但绑定 KVM 且需要 VMX 能力，通用化仍需工作。

06　非特权 VM 的时间共享



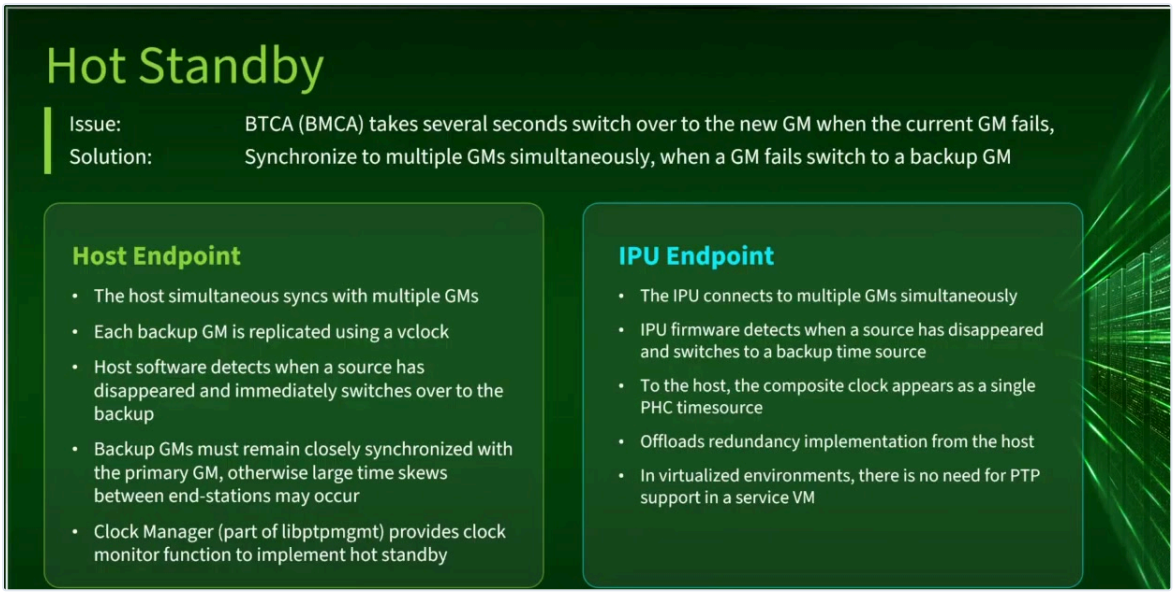
在上一页架构图上新增 UNPRIVILEGED VM：它依据 TSC 与 hypervisor/服务 VM 时钟的关系，在本 VM 内重建出同样的时钟

那么现在，我们加了一个非特权 VM。呃，基本上，我们的做法就和上一页说的完全一样。呃，非特权 VM 知道 TSC 与 hypervisor 上（或者服务 VM 上）那个时钟之间的关系，呃，就能很容易地在自己这个 VM 里重建出那个时钟。

> "And so now, we added an unprivileged VM. [snorts] And uh, basically the I mean, it's we're we're just doing exactly what what we uh, we said on the previous slide. Uh, the uh, the unprivileged VM knows the relationship uh, between TSC and the clock that exists on the hypervisor or the or the service VM and uh, can easily reproduce that clock. In its own in its own VM."

> **小结**　VM 侧无需 PTP 协议栈：只要知道 (TSC, 参考时钟) 的对应关系，就能在本地重建时间。这让"多时间基"对租户透明——租户可自行选择用全局还是机架本地时间。

07　热备份：同时同步多个 GM



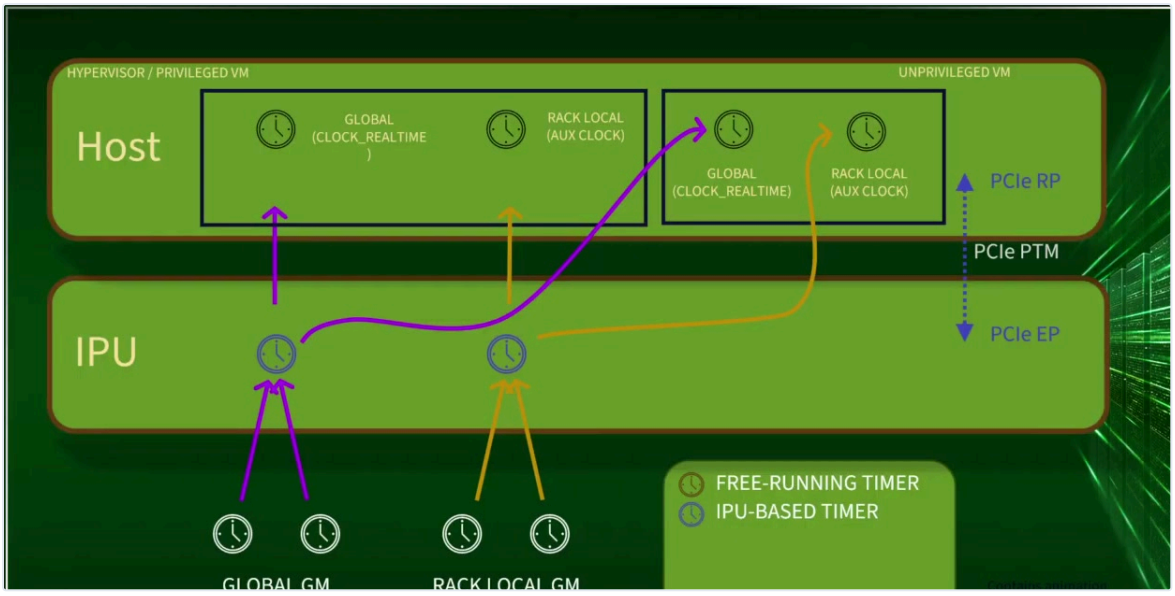
Hot Standby：问题为 BMCA 在当前 GM 失效时需要数秒才能切到新 GM，方案是同时同步到多个 GM，失效时切到备份 GM；左栏 Host Endpoint，右栏 IPU Endpoint

所以，呃，多时钟——多个时间基——就这些。有了机架本地和全局两个时间基。那么现在，我们来讲可靠性。呃，我们提出的是——我已经讲过 BMCA/BTCA 的问题了——我们提出的是用热备份（hot standby）。基本思路是：我们同时同步到多个 GM，当主时间源（主 GM）消失时，我们可以无缝且立即切到另一个时间源。呃，我们看到的实现方式有几种。一种是在主机上做。我们会多讲一点 clock manager——那是我们正在做的东西——可以用它在主机上在主时钟与备时钟之间切换。呃，我们还提出另一种方式，即端点其实在网卡上。这种情况下，所有的冗余、clock manager 的冗余算法，实际上都在网卡本身上，呃，我们把所有这些复杂度从主机挪到网卡上。那么，我们再看一下这在系统层面是什么样子。呃，我们从最基本的 P2P 配置开始：单个 GM、单个……

> "So, um, so that's uh, that's all there is for uh, multiple multiple clocks um, you know, multiple time bases. Got the rack local and uh, global time base. So, now we're going to talk about reliability. And so, we're proposing that I've talked about the problems with BMCA PTCA. So, what we propose is using hot standby. And the basic idea here is that we are synchronized to multiple times multiple GMs at the same time and we can seamlessly and immediately switch over to another time source when when the primary time source goes primary GM goes away. And there's a couple of ways um that we see implementing this hot standby. So, there's one that's the we do this on the host. So, we'll talk a little little bit more about a clock manager. That's something that we're working on. And that's that's something that we can use to switch between the primary and the secondary clock on the host. There's and we propose there's another way to do this also where the endpoint is actually on the nick. And in this case, all of the redundancy the other clock manager redundancy algorithms are actually on the on the on the nick itself and we And basically, we, we move all that complexity from the host to the nick. And so, again, let's take a look at what that looks at a at a system system level. So, we'll start uh uh start with the the most basic P2P configuration, single single GN, single"

> **小结**　两条冗余实现路径：①主机侧——clock manager（libptp management）按主/备时钟切换；②网卡侧——把冗余算法整体下沉到 NIC/IPU，主机只见一个 PHC。后者把复杂度移出主机，是"端点即网卡"的思路。

08　完整图景：冗余、虚拟化与 PTM 卸载



完整架构图：Hypervisor/特权 VM 与非特权 VM 各有 GLOBAL（CLOCK_REALTIME）与 RACK LOCAL（AUX CLOCK）；IPU 同时从两台 GLOBAL GM 与两台 RACK LOCAL GM 取时，并区分 FREE-RUNNING TIMER 与 IPU-BASED TIMER；右侧 PCIe RP / PTM / EP 为时间下发通道

……单个时间基。那么冗余长什么样——同样，我们用 V-clock，但用法不同，因为我们有两个 V-clock：一个代表主时间源，一个代表备时间源。呃，所以，基本上还有一些软件逻辑，这也是我们正在 clock manager 软件里开发的东西。它判断——这一切都在系统层。所以，看 V-clock 和 PTP4L 的状态，就能判断主时间源是否还活着、是否还准确。如果不是，我们就切到备时间源。而且这个冗余还可以和多时间基一起用。所以这大致就是完整图景：我们有冗余的全局时间基和机架本地时间基，再加上虚拟化——虚拟化的运作方式也基本相同。

那么，现在我们讲第二个框，也就是我们把所有这些复杂度卸载到网卡本身。这种情况下，一切都卸载了，所有那些细节对主机都不可见。所以我们只有一个 PHC——呃，此时我们会有多个 PHC，每个时间基一个。呃，没有 P-clock 了。呃，所有这些都透明了。时间——它完全是网卡上的一个端点。而且，呃，我们仍然支持虚拟化，这里我们有更多选择。我们不必在平台内部共享时间。这种情况下，非特权 VM 可以直接从网卡取时间。所以基本上，我们前面讲的所有那些——在主机上做虚拟化和冗余的那些——全部下沉到网卡，呃，不需要主机提供任何支持。

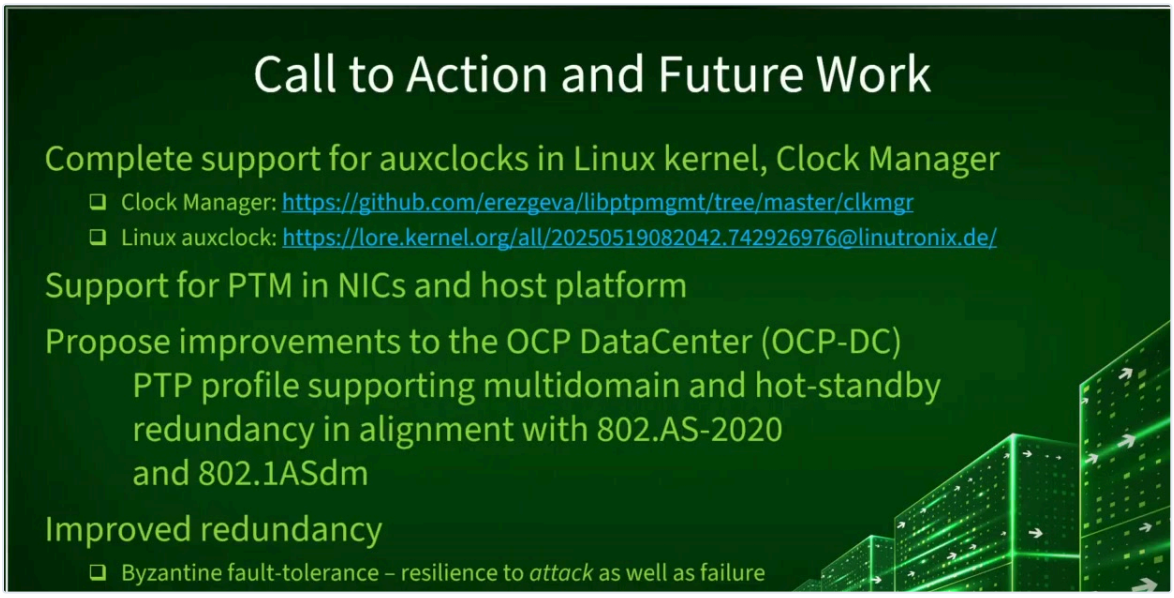
呃，另外，我不打算多讲 PTM，但 PTM 在这里扮演重要角色，呃，在把时间从 IPU（或说 NIC、IPU）传到主机的精度方面。呃，我很高兴今天已经听到 PTM 被提到好几次，这很好。呃，所以我们非常希望看到所有网卡和所有主机都支持 PTM。

> "single time base. And so, what redundancy looks like again, we're using V clocks. We're using them differently here because we have two V clocks, one that would represent say the primary time source, one that represents the secondary time source. Um So, uh And and and basically there's there's some software logic again. This is something with something that we're developing in the the clock manager software. And so, it's determines when one of the And this is all at the all at the system level. So, looking at the V clock and the state of P2P for L determines whether the primary source is still alive and accurate. And if it isn't, then we switch to the secondary time source. So, and this redundancy can also be used along with our multiple time bases. So, this is sort of the complete picture where we have redundant global and rack local time bases. And at virtualization, and basic basically the virtualization works the same way as well.

> So, now we'll go to the the second our our second box where we offload all of this complexity to the network card itself. So, in this in this case again, all of this is offloaded. And now all of that detail is invisible to the host. So, uh we just have a a pH So, we would have at at this point multiple PHCs, one for uh one for each time base. Uh there there's no P clocks. Um all of that is transparent. The the time is uh it's a complete endpoint on on the NIC card. And again, uh we can support virtualization. We have uh we have more options here. We we don't have to we don't have to share the time within the platform. A uh non-privileged VM can directly pull the time from the NIC card in this case. So, uh so basically all all of the all of the stuff that we talked about before that was on on the NIC, you know, with I'm sorry, on the host with virtualization and uh redundancy, um all that is pushed down into the NIC and uh not um uh and uh does not doesn't require any support on the host. So, um and again, um I'm not going to talk a lot about P PTM, but PTM uh plays a big uh big role here um in the accuracy between, you know, transferring time from the IPU to or the NIC IPU to the host. Um and yeah, I'm I'm I'm glad that I've heard uh PTM mentioned several times today. It's uh it's it's good. So, um yeah, we would we would love to see uh PTM supported on all NICs and uh all uh all hosts."

> **小结**　最终形态是"冗余 × 多时间基 × 虚拟化"三者的乘积：主机侧用双 V-clock 表达主/备，网卡侧用多 PHC + 多 GM 直连把冗余做成端点能力，跨 PCIe 用 PTM 保证精度。VM 在网卡卸载模式下可直接取时，无需平台内共享。

09　行动号召与问答



Call to Action and Future Work：补全 Linux 内核 auxclock 与 Clock Manager 支持、推动网卡与主机平台支持 PTM、向 OCP-DC PTP profile 提议多域与热备份冗余改进、提升冗余至拜占庭容错

呃，这就是我今天讲的全部内容。呃，再说一次，我们正在做的 clock manager 软件，是 libptp management 项目的一部分——这个项目存在有一段时间了，允许你通过 1588 管理协议与 PTP 交互。我们使用那套管理协议，并加了一些东西，使我们现在可以在一个源与另一个源之间切换。这是主要的功能点。我还放了 Linux aux clocks 的链接，那个还在开发中，但已经推进得相当远了。呃，我们提议与社区一起推进 OCP 数据中心 profile，我们非常希望得到各位的反馈，把这些特性加进数据中心 profile。谢谢。

（主持人） 谢谢你，Chris，很有意思。我有个问题。在你的 IPU 里，你可能有多个 PTP 时间接收器——如果你有多个 GM 的话。那么你也会有多个 PHC 吗?

（讲者） 呃，是的。我的意思是，它会很像我们在 Linux 上做的。你知道，在 Linux 里我们有这个 V-clock，我们对底层时间戳时钟做软件翻译，然后有一套（切换）机制，在（网卡上）是同样的概念。

（提问者） 谢谢，这回答了我的问题。你是有一个公共的、跑时间戳的公共时钟，然后从它出发的翻译是按你所在哪个 GM 定制的？

（讲者） 对，是这样。

（提问者） 好，谢谢。Stanton 先生。

（提问者） 谢谢。这种情况下你的 GM 是本地还是全局的？

（主持人） 我听不太清，能把他的麦克风调大一点吗？……现在可以了吗？

（提问者） 好。那么这种情况下你有一个全局 GM。

（讲者） 对。

（提问者） 还有一个本地实时 GM。是的。这就是你所说的"多域"吗？如果这就是你指的多域，为什么它会是多域？

（讲者） 你是指 PTP domain，还是别的概念？

（提问者） 对。所以这些都是用 PTP domain 的，但就——我想你问的是机架本地与全局时间基。那些会是——对，那些会是不同的 domain；如果它们是冗余的，那 domain 数量还会相应增加。所以每一个都由自己的 GM 代表，而在机架本地与全局这种情况下，它们是完全不同的时间基。

（提问者） 但是同一个 profile、不同 domain？

（讲者） 抱歉？

（提问者） 我理解是同一个 profile，所以应该是同一个 domain，但你提到的是不同 domain。

（讲者） （确实可以）用多个 domain。所以在 IEEE 1588（PTP 2019）里，我们可以在同一根线上同时跑多个 PTP domain。

（提问者） 好，好。

（提问者） 非常精彩的演讲，谢谢 Chris。你讲到选择时钟、切换时钟。我猜你也可以把所有时钟——不管多少个——直接做集成（ensemble），因为那和前面 Oscilloquartz 讲 resilience 的内容相关。是这样吗？

（讲者） 是的，完全可以那样做，我在这里没有专门讲，但最后一页我们提到了拜占庭容错，所以现在你还可以——你知道，既能保证时间安全，也能应对故障。所以取平均的做法是可行的。我的意思是，我觉得 802.1ASdm 只是主备切换，但当然我们也可以做一些（更复杂的）。

（提问者） 你前面一两页提到软件层，那是你设想中会在软件层支持的吗？

（讲者） 呃，是的。

（提问者） 好，谢谢。

（提问者） 嗨 Chris，很高兴见到你。呃，我有个稍微超前的问题。假设你的测量发现，时间同步在规模上其实不成立，因为——你知道，从量子物理看，时间既向前也向后，甚至横向走。

（讲者） 是的。

（提问者） 我见过一些这样的观察。如果必须放弃时间同步，对你来说什么会不成立？

（讲者） 呃，我的意思是，今天提到了很多应用，但我们在 Intel 特别关注的是 AI 应用和数据库。对于数据库应用，呃，重试率会高得多，呃，我们会有更多冲突。而对于 AI，更多是集群的利用率问题。

（提问者） 为什么事务排序不能解决它？

（讲者） 抱歉？

（提问者） 为什么事务排序不能解决它？

（讲者） 呃，那正是我们从时间里得到的东西。它是一个全局的——所以它是事务的全局排序。所以，时间是实现这一点的一种方式。

（提问者） 为什么你不能造一个——你知道，随便什么源，只要能给事务排序，而时间不是那个源？我只是问一个很基本的问题，你不必回答，但可以想一想。

（讲者） 好。

（提问者） 好，就这些。谢谢。

（讲者） 谢谢。

（主持人） 谢谢各位。

> "So, um that's uh that concludes what I've talked about today. And um so, again, the uh clock manager software that we're working on, that's part of the libptp management project, which is something that's been around for a while that allows you to interface with with PTP using 1588 management protocol. So, we use that management protocol and add some more stuff that we can now now switch from um one source to another. That's the that's the major kind of feature there. And I've also included a a link to the Linux aux clocks, which are uh which are which are a little bit in development. They're quite a way along. And uh what's what we're proposing is to work work with the community on the OP OCPE data center profile, and we uh we love to get any of your feedback, and uh yeah get uh add add uh add these features to the data center profile. Thank you, Chris. That was a lot of fun. I had a question. In your IPU, you may have multiple uh time PTP time receivers. If you have multiple GMs, Yeah. So, you do you have multiple PHCs as well? Um yeah. And I mean, yeah, it would it would look a lot like um it would look a lot like what we have on on Linux. So, you know, in Linux, we have this V clock where we have a software translation of the underlying timestamp clock and we'd have the set and it's the same concept in the Thank you. That that answered my question. You have a You have a something common, a common clock which is running the time stamp and then the translation from that is is custom to which GM you are That's right. Yeah. Okay. Thank you. Mr. Stanton. All right. Thank you. In this case you GM local or global I'm having I'm having some trouble. Can we get his mic turned up? Yeah. Yeah. Are you okay now? Yeah. Yeah. So in this case you have a global GM That's right. and local real time GM Yes. Is this multiple domain as you refer to or is it If this is what you refer to why would be multiple domain? So I you mean the domain is a PTP domain or is different concept in your Yeah. So I'm So all of these all of these use PTP domains but in terms of So they I mean I think you're asking about the rack local versus the global time base. And those would be Yeah. Those would be separate domains and if they're redundant then there's that many domains as well. So each of these would be represented by their own GM and in the case of the rack local versus global they would be totally different time bases. But you same

> profile is different domain or I'm sorry. I understand it's the same profile so they should be same domain but you mentioned different domain. to use multiple domains. So with uh So with with IEE 2000 you know PTP 2019 we could have multiple PTP domains um running simultaneously over the same wire. Okay. Okay. Uh great great presentation. Thank you Chris. You talked about selecting a clock, changing clock. I guess though you could also just take all of the clocks however many there were and then somehow ensemble them as well because that which relates back to the previous to the solar quartz talk about resilience and and all of that. Is that right? Yes. Yeah. You could definitely do that and I didn't specifically call it out here but on the last slide we talked about Byzantine fault tolerance and so now we also you know can you know secure the time as well as yeah. So your way of averaging is is possible. I mean I think that one AS DM is just sort of a primary secondary switches but certainly it could be we could do some of And you mentioned the software layer I think a slide or two before but is that something that would be supported in the software layer as you envision it now? Um yeah. Okay. Thank you. Hey Chris. Good to see you. Um So, I have a question that's slightly in the future. Suppose you know, measurements you find out that time synchronization actually doesn't work uh at scale because uh you know, from quantum physics time goes both forward and backward and literally goes sideways. Yeah. I have seen some of these observations. What is it that will not work for you if you had to get rid of time synchronization? Um so, I mean, there's there were a lot of applications mentioned today, but what we're looking you know, specifically at Intel, what we're really looking at are AI and um uh AI applications and uh and database. So, with with database applications, uh the retry rate is is going to be you know, much higher. Uh we're going to have a lot of have more more collisions. And uh for you know, AI, it's more about uh utilization of the of the cluster. Why wouldn't order transaction solve that? Excuse me? Why would transaction ordering not solve it? Um well, that's what's that's what we're getting out of out of time. So, it's a it's a global So, it's a global ordering of the transactions. So, I mean, time is the way is is a way to do that. Why couldn't you make it a you know, fake um you know, any any source for anything that just orders transactions and time is not the source? I I'm just asking a very basic question. You don't have to answer, but think about it. Yes. Okay, that's it. Thank you. Thank you. Thanks, guys."

> **小结**　四项后续工作：内核 auxclock 与 Clock Manager 落地、PTM 在网卡与主机的普及、把多域与热备份写进 OCP-DC PTP profile、把冗余从主备升级到拜占庭容错。问答补充了三点关键澄清：多时间基对应不同 PTP domain（可在同一线缆并行）、冗余可替换为多源集成（ensemble）、网卡侧多 GM 对应多 PHC 并用公共时间戳时钟做翻译。

附录　时间轴

时间	内容
00:00	开场，说明网络与系统两条线
00:21	精度与可用性两个目标；BMCA 倒换窗口问题
01:21	全局 vs 机架本地时间尺度；每链路约 4ns
02:54	系统架构图：V-clock、aux clock、自由运行时钟
05:09	时间虚拟化：以 TSC 为公共基准
06:39	非特权 VM 重建时钟

时间	内容
07:30	热备份：主机侧与网卡侧两种实现
09:30	完整图景：冗余 + 多时间基 + 虚拟化 + PTM
13:18	行动号召
13:50	问答（多 PHC、PTP domain、ensemble、时间同步的必要性）
21:17	结束